院　系 School of Computer and Artificial Intelligence　学　号 2024220023, 2025220047

姓　名 Al Refat, A. K. Fahim

西南交通大学研究生 2025－2026 学年第(2)学期考试

课程代码 GS14805004 课程名称 云计算与边缘智能原理及应用

任课教师签字： 戴朋林、李洋、陈香伊

一．考核要求：

Assessment is a semester-long team project (2-3 students per team): design and implement a cloud-edge-device collaborative intelligent application system (e.g. intelligent video surveillance, industrial IoT anomaly detection, smart traffic analysis, edge-assisted Q&A, medical data processing, federated learning, cloud-edge recommendation, or a self-proposed topic). The work has three progressive phases, each graded independently by one instructor; the technology stack is free and open-source simulation tools are encouraged.

Phase 1 - System Design & Cloud Platform (Weeks 1-6): a clear three-layer (cloud/edge/device) architecture with well-defined data flow and interfaces, plus runnable core cloud services (data storage, APIs, business logic).

Phase 2 - Edge Intelligence Deployment & Optimization (Weeks 7-12): select or train a suitable model and apply at least one optimization technique (quantization, pruning, distillation, or lightweight design); deploy inference under resource constraints and compare accuracy, latency, and resource cost.

Phase 3 - Cloud-Edge Collaboration & Integration (Weeks 13-17): build an end-to-end pipeline and at least one collaboration strategy (task offloading, data synchronization, model update, or privacy protection); evaluate end-to-end latency, throughput, and resource utilization under varying conditions.

Deliver a technical report (single column, Times New Roman 11 pt, single line spacing, with references), runnable source code with deployment documentation, and an oral defense.

二．评分标准：

1. 三层（云/边/端）架构清晰，云平台、边缘智能与云边协同功能完整、端到端稳定运行；边缘优化得当、对比实验充分（准确率、时延、资源消耗分析深入），协同策略合理且有理论与文献支撑；报告规范、图表与文献综述详实，演示逻辑清晰、答辩专业深入。（>90 分）

2.架构与三个阶段实现较完整、系统基本可端到端运行；边缘优化与对比实验合理、云边协同策略可行；对相关文献了解较全面并能批判性分析；报告条理清晰、图表得当，演示清楚、答辩较有条理。（80-90 分）

3.主要功能基本实现但完整性一般，实验或协同策略较简单；报告有少量拼写、语法或格式问题；对文献有所了解，演示较清晰、基本守时，能回答主要问题。（70-79 分）

4.系统不完整或集成存在明显问题、实验不充分；报告拼写和语法错误较多，影响清晰度；文献覆盖不足，演示清晰度或时间控制不佳。（60-69 分）

5.系统无法运行或核心交付物缺失；报告存在抄袭或严重语法错误；未按要求提交代码与报告，或不出席演示与答辩等。（0-59 分）



云计算和边缘智能原理及应用

Cloud Computing and Edge Intelligence Principles and Applications

建筑设计报告

Architecture Design Report

课题名称 Project Name	A Lightweight, Subscription-Free Home Perimeter Monitoring System

Table of Contents

Abstract 5
Chapter 1: Introduction 6
1.1 Motivation and Problem Statement 6
1.2 Existing Open-Source Solutions and Their Gaps 6
1.3 Research Objectives 7
1.4 Report Structure 7
Chapter 2: Literature Review 7
2.1 Object Detection for Surveillance 7
2.2 Multi-Object Tracking 8
2.3 Open-Source NVR Platforms: ZoneMinder and Frigate 8
2.4 Inference Optimization and TensorRT 9
2.5 Edge Computing and Containerization 9
2.6 Alert and Notification Channels 9
2.7 Streaming and Control Interfaces 10
Chapter 3: System Architecture 10
3.1 Architectural Overview 10
3.2 Per-Camera Worker Architecture 13
3.3 Data Flow 13
3.4 API Layer and Dashboard 14
Chapter 4: Implementation 16
4.1 Configuration and Model Abstraction 16
4.2 Model Warm-Up 16
4.4 Intrusion State Machine 17
4.5 MJPEG Streaming 19
4.6 Telegram Notification Subsystem 19
4.7 Database Schema and Operations 20
4.8 Containerization and GPU Passthrough 20
Chapter 5: Experimental Setup and Results 20
5.1 Hardware and Software Environment 20
5.2 Inference Backend Benchmark 21
5.3 Per-Module Latency Breakdown 21
5.4 Multi-Stream Resource Usage 22
5.5 Multi-Device Access Validation 22
Chapter 6: Tested Deployment Scenarios 22
6.1 Backyard Perimeter Monitoring 22
6.2 Main Gate / Entry Point Monitoring 23
6.3 Indoor Doorway Monitoring 24
6.4 Toddler/ Children Monitoring (Repurposed Use Case) 24

6.5 Limitations 25
Chapter 7: Conclusion 25
References 28

Abstract

This paper presents the design and implementation of a containerized, GPU-accelerated perimeter monitoring system targeted at residential and small-scale commercial deployments. The system integrates YOLO11s, a state-of-the-art single-stage object detector released by Ultralytics in 2024 [1], with the ByteTrack multi-object tracker [2] to detect and continuously track persons within configurable polygon regions of interest (ROIs). A FastAPI back-end [3] exposes a REST/MJPEG interface over a local network, enabling any on-network device to view live annotated camera streams, interact with a web dashboard, and receive intrusion alerts via the Telegram Bot API [4]. Persistent intrusion records are stored in an embedded SQLite database, and the entire application runs as a Docker container [5] with NVIDIA GPU passthrough, requiring no third-party subscription or proprietary cloud infrastructure. Unlike established open-source NVRs such as ZoneMinder [6] and Frigate [7], which rely on motion-triggered detection pipelines and tighter ties to specific home automation ecosystems, the proposed system performs continuous zone-aware person detection with configurable ROI polygons and exposes a fully self-contained REST API that operates independently of any automation hub. Benchmarks on an AMD Ryzen 7 7840H laptop with an NVIDIA RTX 4060 GPU show that Direct TensorRT inference achieves a mean pipeline latency of 7.61 ms per frame (131 estimated FPS), a 2.37x improvement over native PyTorch at 18.06 ms (55 FPS). Running two simultaneous streams, total VRAM consumption remains near 1.1 GB, a footprint compatible with NVIDIA Jetson modules or AI-accelerated Raspberry Pi deployments [8].

Keywords: Home Surveillance, Edge-AI Security System, Multi-object Tracking, Perimeter Intrusion Detection, Subscription-free Monitoring.

Chapter 1: Introduction

1.1 Motivation and Problem Statement

Residential and perimeter security has traditionally relied on closed-circuit television (CCTV) ecosystems that route footage to proprietary cloud services for processing and storage. These services impose recurring subscription fees, introduce network latency, and transfer sensitive visual data to third-party servers, raising significant privacy concerns [9]. Beyond the residential context, intelligent surveillance has been applied to critical infrastructure such as railway level crossings, where computer-vision-based trespass detection has been shown to meaningfully reduce human monitoring burden and improve response times [10][11]. Public space surveillance in pedestrian-dense environments such as train stations, platforms, and event venues similarly benefits from automated person detection and zone-based alerting [12]. These applications share a common architectural challenge: real-time inference on resource-constrained or embedded hardware, without dependence on cloud processing.

Consumer-grade edge devices have reached a level of computational maturity, particularly with GPU-equipped modules such as the NVIDIA Jetson family [8], that makes on-premise AI inference not only feasible but cost-effective for small deployments. The proliferation of powerful single-stage object detectors, beginning with the original YOLO architecture [13] and continuing through YOLO11 [1], has dramatically reduced the compute cost of detecting persons in real-time video. Paired with efficient multi-object trackers such as ByteTrack [2] and the annotation utilities provided by Roboflow Supervision [14], a complete and low-latency person detection pipeline can be assembled entirely from open-source components.

1.2 Existing Open-Source Solutions and Their Gaps

Two open-source surveillance platforms are widely adopted in the homelab community: ZoneMinder [6], a Linux-based NVR first released in 2002, and Frigate [7], a modern Docker-native NVR that integrates with Home Assistant. ZoneMinder provides comprehensive multi-camera management, motion-triggered recording, and a web interface, but its detection engine relies on classical motion-pixel analysis rather than deep-learning object recognition. Its deployment footprint is large: the official container bundles MySQL, Apache, and the ZoneMinder daemon in a single image, and its resource consumption on multi-camera workloads is high compared to purpose-built inference pipelines. Frigate addresses these shortcomings by offloading object detection to Google Coral TPU, Hailo, or NVIDIA GPU accelerators using TensorFlow Lite and YOLO-class models, and by using a motion-pre-filter to reduce the number of frames sent to the detector. However, Frigate is tightly coupled to the MQTT and Home Assistant ecosystem, requires a separate broker, and does not expose an inference-level REST API that can be consumed independently.

The system described in this paper occupies a complementary niche: a purpose-built, single-container application that exposes a full-featured REST API and MJPEG streaming endpoint, performs continuous (not motion-gated) YOLO11s inference with ByteTrack tracking, evaluates detections against user-defined polygon zones, and sends alerts via Telegram without requiring any auxiliary services. This design prioritizes simplicity of deployment, configurability, and network-accessible control from any browser.

1.3 Research Objectives

This work contributes the following:

1. A layered, thread-per-camera architecture that decouples frame capture, YOLO11s inference, ROI zone evaluation, JPEG encoding, and MJPEG streaming into independent timing domains, enabling each to run at its optimal rate.
2. A polygon-based ROI enforcement mechanism built on Supervision PolygonZone [14] that confines alerts to user-defined scene regions, reducing false positives from activity outside areas of interest.
3. A non-blocking Telegram notification subsystem that dispatches snapshots and text alerts through a background thread, ensuring network latency never degrades inference throughput.
4. Dual model backend support (PyTorch .pt and TensorRT .engine) with measured speedup documented across three inference backends on a consumer RTX 4060 GPU.
5. A fully browser-native control dashboard and interactive ROI editor requiring no native client installation, enabling any on-network smartphone or laptop to manage cameras and view live streams.

1.4 Report Structure

Chapter 2 surveys related work across object detection, multi-object tracking, inference optimization, edge computing, and relevant open-source NVR platforms. Chapter 3 describes the system architecture and data flow. Chapter 4 details implementation of each component. Chapter 5 presents experimental setup, benchmark data, and results. Chapter 6 describes tested deployment scenarios. Chapter 7 discusses limitations and concludes the paper.

Chapter 2: Literature Review

2.1 Object Detection for Surveillance

The YOLO family of detectors originated with Redmon et al. [13], who reframed object detection as a single regression problem solvable in one forward pass, eliminating the two-stage proposal mechanism of R-CNN variants [15][16]. Successive revisions through YOLOv8 iteratively refined anchor strategies, backbone design, and training procedures [17][18][19]. YOLO11, released in September 2024, introduced a C3k2 cross-stage partial bottleneck and a C2PSA spatial-attention module, achieving higher mAP on COCO at fewer parameters than its predecessor [1]. The small variant YOLO11s is particularly relevant to edge deployments due to its low VRAM footprint [8].

Surveillance-specific studies have validated YOLO-class detectors across diverse scenarios. Han et al. demonstrated real-time person detection on IP cameras using YOLOv5, reporting acceptable accuracy at 15-20 FPS on embedded GPU modules [20]. Zhu et al. compared YOLOv7 against earlier detectors on the VIRAT outdoor surveillance dataset and concluded that single-stage detectors are practically superior to two-stage alternatives on resource-constrained hardware [21]. Chen et al. applied YOLOv11-Seg to safety monitoring at construction sites, demonstrating robustness under partial occlusion and challenging lighting [22].

Beyond residential security, YOLO-class models have found critical application in public infrastructure. At railway level crossings, RSK Business Solutions trained multiple YOLO variants on over 42,000 annotated crossing images to detect pedestrians entering danger zones, achieving reliable detection during live pilot deployments at three high-risk crossings in southern England [10]. A US Federal Railroad Administration study used Mask RCNN to detect trespass violations at highway-rail

grade crossings [23], while subsequent ensemble approaches using YOLOv8 variants improved detection of dynamic crossing objects including pedestrians, buses, and trains [24]. In dense urban environments such as transit stations and public plazas, deep learning pedestrian detection has been applied to crowd safety control, density estimation, and abnormal behavior detection [12], establishing the broad applicability of the same core detection architecture used in this work.

2.2 Multi-Object Tracking

Person detection in static frames is insufficient for accurate intrusion logic: a fleeting detection from sensor noise must be distinguished from sustained presence. Multi-object tracking (MOT) provides persistent identity assignment across frames. SORT [25] predicts object positions via a Kalman filter [26] and resolves assignment with the Hungarian algorithm [27], but discards low-confidence detections, producing fragmented trajectories. ByteTrack [2], presented at ECCV 2022, performs a second association pass on low-score detections using IoU similarity with existing tracklets, recovering true objects scored below threshold. ByteTrack achieved 80.3 MOTA and 77.3 IDF1 on MOT17 at 30 FPS. Railway network surveillance surveys have documented the use of extended Kalman filters and Gaussian mixture models for tracking moving objects across frames in security-critical infrastructure [28], further validating the need for robust multi-frame association. Alternative trackers such as DeepSORT [29] add a deep appearance descriptor for re-identification, beneficial in crowded scenes but adding compute cost unnecessary for residential perimeter monitoring.

2.3 Open-Source NVR Platforms: ZoneMinder and Frigate

ZoneMinder [6], created by Philip Coombes and first released in 2002, is the longest-established open-source Linux surveillance platform. It provides an integrated suite for capture, analysis, recording, and monitoring across IP, USB, and analog cameras, with a web-based GUI for remote access and email alerting. ZoneMinder's zone-based motion detection allows users to partition camera frames into regions with different sensitivity thresholds, a conceptual predecessor to the polygon ROI approach used in this work. However, its detection mechanism remains pixel-difference motion analysis rather than deep-learning object recognition, which requires extensive manual tuning to suppress false positives from lighting changes, wind-blown foliage, and animal movement. Its deployment complexity is also higher: the official installation bundles Apache, MySQL, and Perl-based daemons in a single system image, making containerization non-trivial [6].

Frigate NVR [7], developed by Blake Blackshear and released as open source under the MIT license, represents the modern generation of self-hosted NVR. It is designed specifically for Docker deployment and Home Assistant integration, using MQTT for event publication and TensorFlow Lite for object detection. Frigate employs a two-pass pipeline: a lightweight motion detector identifies regions of interest within each frame, and the object detector is applied only to those regions, dramatically reducing inference load. This architecture allows Frigate to achieve over 100 object detections per second on a Google Coral Edge TPU. Frigate also supports NVIDIA CUDA and TensorRT inference, Intel OpenVINO, and Hailo accelerators. Its tight coupling to the Home Assistant ecosystem and MQTT broker is a strength for smart-home integrations but represents an additional infrastructure dependency for deployments that do not use Home Assistant. The proposed system removes this dependency while adding REST-native control and Telegram-native alerting.

Table 2.1 summarizes the key differences between ZoneMinder, Frigate, and the proposed system across dimensions relevant to the design objectives.

Table 2.1. Comparison of Open-Source Surveillance Platforms

Feature	ZoneMinder	Frigate	This Work
Detection Engine	Pixel motion analysis	TFLite / YOLO (Coral, GPU)	YOLO11s + ByteTrack (GPU)
Zone Configuration	Sensitivity zones	Object zones (polygon)	Polygon ROI editor (browser)
Deployment	Native Linux / Docker (complex)	Docker Compose	Docker Compose (single container)
Alert Channel	Email / SMTP	MQTT / Home Assistant	Telegram Bot API
External Dependencies	MySQL, Apache, Perl	MQTT broker, Home Assistant	None (self-contained)
Stream Access	MJPEG (web UI)	RTSP re-stream + web UI	MJPEG REST API (any browser)
Inference Mode	Motion-gated (pixel diff)	Motion-gated (pre-filter)	Continuous, time-gated

2.4 Inference Optimization and TensorRT

NVIDIA TensorRT [30] transforms a trained network graph into a serialized engine optimized for a specific GPU target through layer fusion, kernel auto-tuning, and FP16/INT8 precision reduction. The Ultralytics YOLO11 export pipeline produces a TensorRT engine from a .pt checkpoint with a single command. Benchmarks in Section 5 confirm that Direct TensorRT inference on the RTX 4060 achieves 2.80x faster model wall time and 2.93x faster raw inference than native PyTorch. FP16 half-precision inference, enabled via half=True in both backends, roughly halves VRAM consumption and increases CUDA tensor-core utilization, enabling two independent streams within an 8 GB VRAM envelope.

2.5 Edge Computing and Containerization

Edge computing relocates computation from centralized data centers to devices physically close to data sources, reducing round-trip latency and eliminating WAN dependency [31]. For video surveillance this is critical: streaming video to a cloud endpoint introduces latency of hundreds of milliseconds and creates bandwidth demands incompatible with residential internet connections [32]. Docker [5] decouples application code from the host OS by packaging code, runtime, libraries, and configuration into a portable image. NVIDIA Container Toolkit enables GPU passthrough to containers, permitting full CUDA and TensorRT access. Empirical evaluation confirms Docker introduces negligible overhead for compute-bound inference workloads while significantly simplifying deployment and reproducibility [33][34].

2.6 Alert and Notification Channels

Early surveillance systems delivered alerts via email or SMS, which impose latency and per-message fees. The Telegram Bot API [4], introduced in 2015, provides a free HTTP interface for sending text messages, photos, and videos to users or groups. Its platform-agnostic reach (iOS, Android, Windows, macOS, Linux, web) and MTProto end-to-end encryption make it well-suited for IoT alert delivery. Prior work has validated Telegram bots for infrastructure monitoring [35], access control notification [36], and IoT sensor alerts [37]. A key implementation requirement is non-blocking dispatch: the inference thread must never wait on a network send. The TelegramNotifier component described in Section 4 addresses this with a bounded background queue.

2.7 Streaming and Control Interfaces

The MJPEG streaming technique, delivering frames as a multipart/x-mixed-replace HTTP response [38], has been used since the early internet era to stream video in standard browsers without plug-ins. FastAPI/Uvicorn [3][39] can serve MJPEG streams alongside JSON control endpoints within a single process, eliminating inter-process communication and simplifying deployment. SQLite [40] provides serverless embedded event logging with zero configuration, adequate for write rates far above the intrusion event rate of a residential deployment [41].

Chapter 3: System Architecture

3.1 Architectural Overview

The system is organized into four horizontal layers that map to distinct runtime phases. The Data Acquisition Layer ingests raw video frames from camera sources. The Inference and Analysis Layer performs YOLO11s detection, ByteTrack tracking, polygon zone evaluation, and intrusion state management. The Persistence and Notification Layer records events to SQLite and dispatches Telegram alerts. The Presentation Layer encodes annotated frames to JPEG, streams them over HTTP as MJPEG, and serves the REST API. Each layer executes concurrently within a Python daemon thread; inter-layer communication uses lock-protected shared state and a threading.Condition for streaming synchronization.

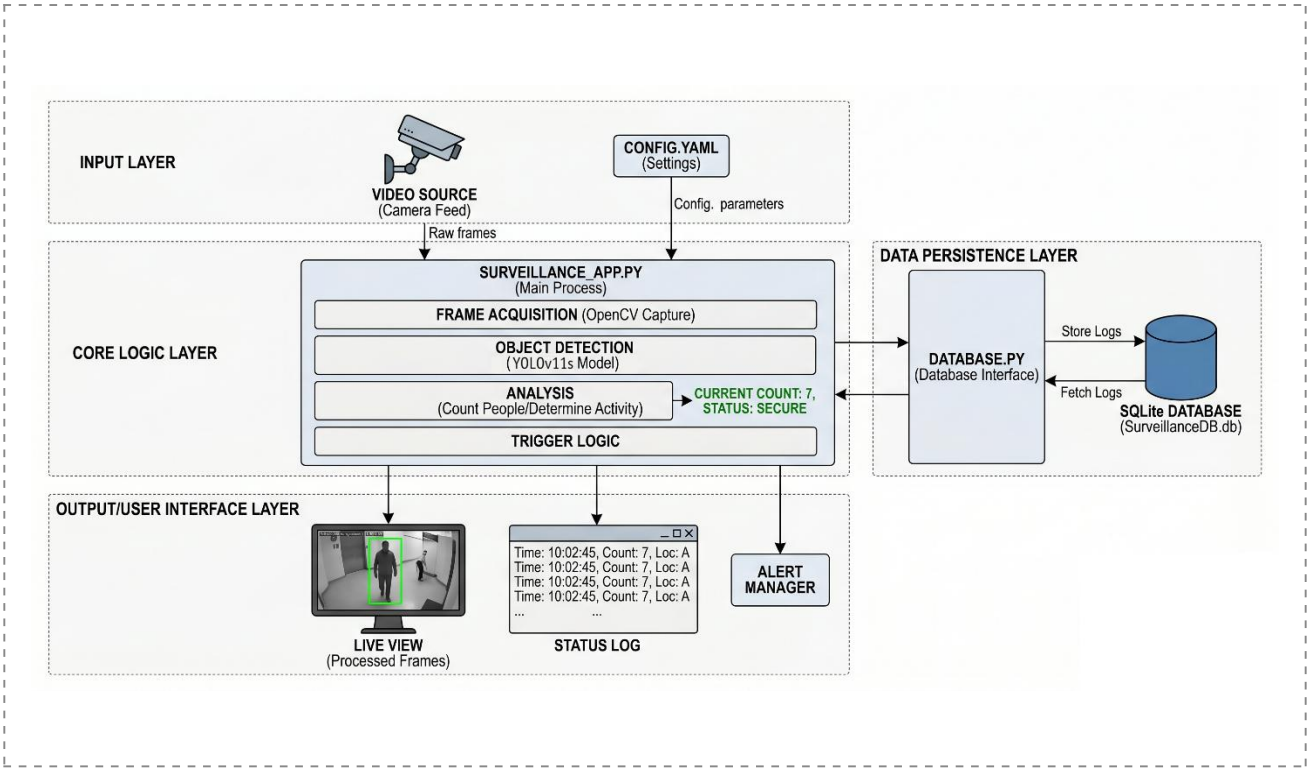


Fig. 3.1. Low-level four-layer architecture of the proposed surveillance system.

The four-layer architecture depicted in Fig. 3.1 reflects a deliberate design choice that separates concerns according to temporal and computational characteristics. The Data Acquisition Layer operates at the camera's native frame rate, typically 30 FPS for standard IP cameras, and is optimized for minimal latency through OpenCV's CAP_FFMPEG backend, which leverages hardware-accelerated decoding where available. This layer's primary responsibility is to maintain a fresh frame buffer without

introducing backpressure that could cause the camera's internal buffer to overflow, a common failure mode in poorly designed surveillance pipelines.

The Inference and Analysis Layer represents the computational heart of the system. Unlike motion-gated approaches that sparsely invoke the detector, this layer performs continuous inference at a configurable rate (default 120 Hz). The decision to decouple inference rate from capture rate stems from two observations: first, modern GPU-accelerated detectors can process frames faster than most cameras can supply them; second, maintaining a consistent inference cadence simplifies timing analysis and ensures deterministic behavior under varying scene complexity. The layer's internal concurrency model uses monotonic time-gating rather than additional threads, a design choice that eliminates locking overhead between the capture and inference paths while guaranteeing that no single slow operation blocks the pipeline.

The Persistence and Notification Layer handles all stateful operations including SQLite commits, video file writes, and Telegram API calls. All I/O-bound operations are delegated to background threads, ensuring that the inference thread never blocks on disk or network latency. The bounded queue design (maxsize=32) provides backpressure handling during network outages, preventing unbounded memory growth while maintaining alert delivery guarantees for recent events. The SQLite schema is optimized for the query patterns expected in a residential deployment: chronological retrieval of events by camera, which is indexed by both camera_id and start_ts.

The Presentation Layer completes the architecture by providing browser-native interfaces for both human viewing and programmatic control. The MJPEG streaming endpoint uses FastAPI's StreamingResponse with a threading.Condition synchronization primitive, enabling fan-out to multiple concurrent clients without per-client encoding overhead. Each client receives the same pre-encoded JPEG frame, reducing CPU utilization to approximately 1-2% per additional viewer. The REST API follows resource-oriented design principles, exposing cameras, configuration, and logs as addressable resources with appropriate HTTP methods.

Table 3.1. System Layer Summary

Layer	Primary Components	Concurrency Model	Key Technology
Data Acquisition	cv2.VideoCapture, CAP_FFMPEG	Blocking read in worker thread	OpenCV 4.x [42]
Inference & Analysis	YOLO11s, ByteTrack, sv.PolygonZone	Time-gated in worker thread	Ultralytics, Supervision [1][14]
Persistence & Notification	IntrusionDB, TelegramNotifier	SQLite commit; daemon thread	sqlite3, requests [40][4]
Presentation	MJPEG generator, FastAPI REST	Async ASGI via Uvicorn	FastAPI, Uvicorn [3][39]

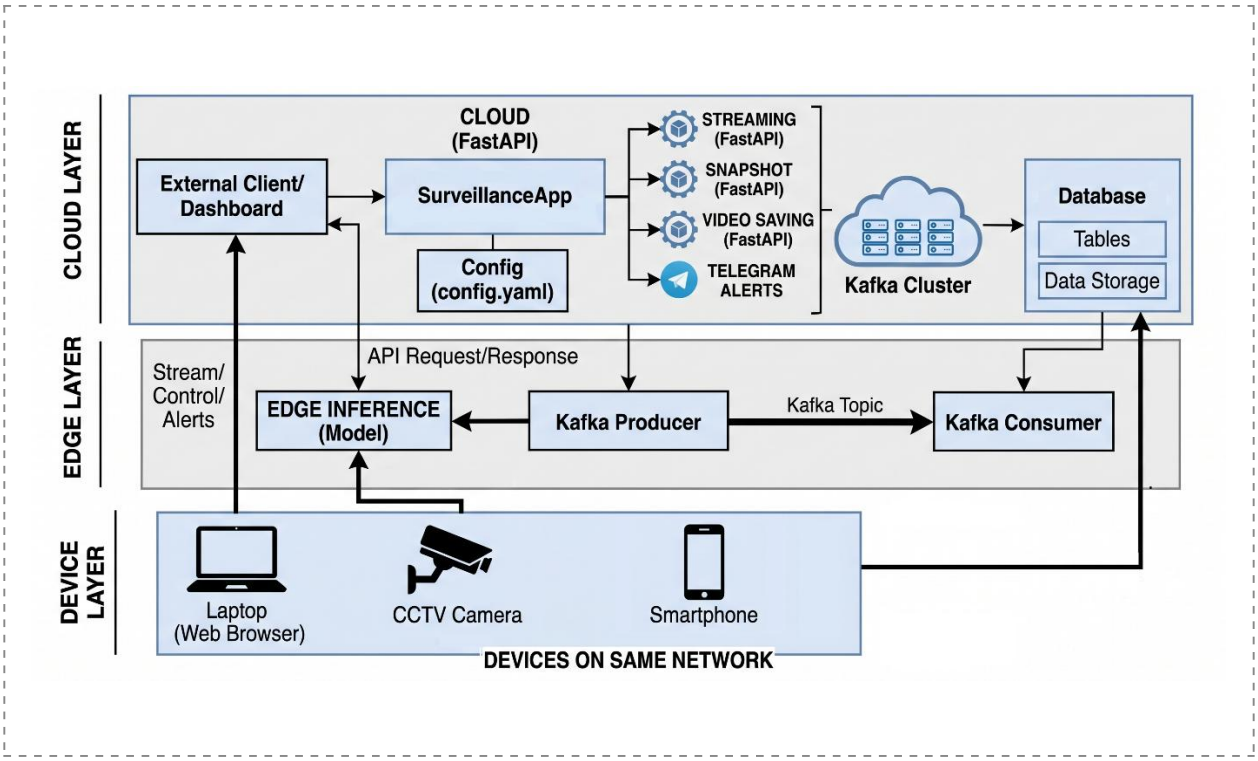


Fig. 3.2. High-level cloud/edge/device layer architecture of the proposed surveillance system.

The cloud/edge/device decomposition shown in Fig. 3.2 aligns the system with contemporary edge-AI architectural patterns while eliminating dependencies on cloud infrastructure. At the device layer, IP cameras and optionally NVIDIA Jetson modules serve as data sources. The camera layer is deliberately stateless, with all intelligence pushed to the edge layer where the Docker container runs. This architecture choice reflects a broader trend in edge computing: rather than simply moving cloud workloads to the edge, the system rethinks the distribution of responsibilities between tiers.

The edge layer, hosted on a local GPU-equipped machine or embedded module, performs all inference, tracking, zone evaluation, and persistence. This concentration of computational responsibility at the edge provides four key advantages over cloud-centric alternatives: (1) sub-100 ms end-to-end latency for alert generation, (2) complete data sovereignty with no video frames leaving the premises, (3) operation during internet outages with alerts cached and replayed upon reconnection, and (4) predictable cost structure with no per-event or per-stream subscription fees.

The user layer encompasses any device capable of running a modern web browser, including smartphones, tablets, laptops, and desktop workstations. By using standard HTTP and MJPEG, the system avoids native application requirements and operates across operating systems without modification. The Telegram notification channel provides a complementary interaction model, delivering rich alerts (images with captions) directly to users' preferred messaging application, reducing the friction of checking the dashboard for every event.

Critically, the architecture's data flow is entirely local: video frames never leave the edge device, snapshots and recordings persist on local storage, and all API requests remain within the local network. This design satisfies data locality requirements that are increasingly mandated by privacy regulations including GDPR and CCPA, which impose strict constraints on transferring personal data (including surveillance footage containing identifiable individuals) to external processing facilities.

3.2 Per-Camera Worker Architecture

Each configured camera is represented by a CameraWorker object owning exactly one Python daemon thread. Within that thread, four operations are interleaved using monotonic time-gating rather than additional threads or asyncio coroutines, avoiding inter-thread queue overhead while ensuring no subsystem blocks the pipeline:

- Frame Capture runs continuously, draining the capture buffer at source FPS to prevent back-pressure accumulation.
- Inference runs at the configured process_fps ceiling (default 120 Hz), controlled by comparing time.perf_counter() to a next_infer deadline.

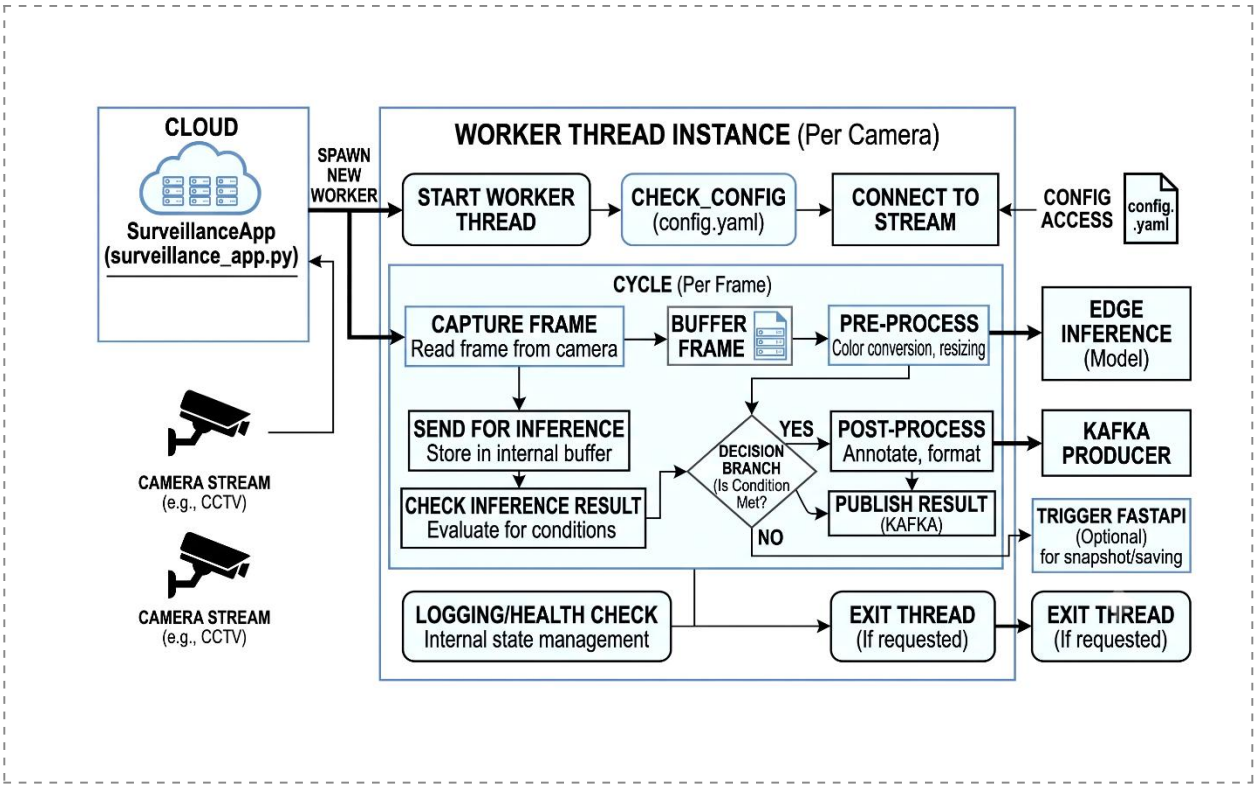


Fig. 3.3. Per-camera worker thread execution flow and timing domain separation.

- MJPEG Encoding and Publishing runs at stream_fps (default 100 Hz), encoding the annotated frame to JPEG and signaling waiting HTTP clients via threading.Condition.
- Video Recording writes annotated frames to an mp4v-encoded VideoWriter at record_fps (default 12 FPS).

3.3 Data Flow

Raw BGR frames are read from cv2.VideoCapture using CAP_FFMPEG. After capture, frames enter the time-gated inference path, where model.track() invokes YOLO11s with ByteTrack. The Ultralytics result is converted to a Supervision Detections object, and sv.PolygonZone.trigger() evaluates each bounding box centroid against the configured polygon. If in-zone person count transitions from zero to positive, the state machine enters ALERTED: a snapshot JPEG is saved, a VideoWriter is opened, an SQLite row is inserted, and a Telegram job is enqueued. When the zone stays empty for patience_frames cycles, the machine returns to NORMAL, the VideoWriter is released, and the SQLite row is closed with a computed duration. The annotated frame is then conditionally JPEG-encoded and pushed to latest_jpeg, triggering notify_all() on the condition variable.

The data flow pipeline can be formalized as a sequence of transformations applied to each frame. Let the raw BGR frame at time t be denoted as $I_t \in \mathbb{R}^{H \times W \times 3}$. The frame capture operation performs a blocking read from the camera buffer: $I_t = \text{cap.read}()$. The inference path is activated at discrete times t_i satisfying $t_i \geq t_{i-1} + \Delta_t$, where $\Delta_t = 1/f_{\text{infer}}$ is the inference interval determined by the configured process fps ceiling.

When activated, the YOLO11s detection function D produces a set of bounding boxes with class probabilities and confidence scores:

$$\mathcal{B}_t = D(I_t) = \{(b_k, c_k, s_k)\}_{k=1}^{N_t}$$

where $b_k \in \mathbb{R}^4$ represents the normalized bounding box coordinates, $c_k \in \mathcal{C}$ is the predicted class (with $\mathcal{C} = \{\text{person}\}$ in this system), $s_k \in [0,1]$ is the confidence score, and N_t is the number of detections at frame t.

The ByteTrack tracker T maintains a set of active tracklets $\mathcal{T}_t = \{\tau_j\}_{j=1}^{M_t}$, each with an associated Kalman filter state [26]. Association is performed in two stages: high-confidence detections ($s_k > \tau_{\text{high}}$) are matched to existing tracklets using IoU-based Hungarian assignment, while low-confidence detections ($\tau_{\text{low}} < s_k \leq \tau_{\text{high}}$) undergo a second association pass. This dual-stage mechanism, formalized in the ByteTrack paper [2], addresses the common failure mode of discarding true positives that are temporarily occluded.

The zone evaluation function Z maps each detected bounding box centroid $p_k = (x_k, y_k)$ to a boolean indicating whether the point lies within the user-defined polygon P:

$$Z(p_k) = \begin{cases} 1 & \text{if } p_k \in P \\ 0 & \text{otherwise} \end{cases}$$

The polygon membership test is implemented using the ray casting algorithm [11], which determines point-in-polygon status in $O(|P|)$ time where $|P|$ is the number of polygon vertices. For a residential deployment with $|P| \leq 20$, this test introduces negligible computational overhead (<0.01 ms per detection).

The intrusion state machine S transitions between states based on the count of in-zone persons:

$$n_t = \sum_{k=1}^{N_t} Z(p_k) \cdot \mathbb{1}[c_k = \text{person}] \cdot \mathbb{1}[s_k > \tau_{\text{conf}}]$$

When n_t transitions from 0 to positive, S enters the ALERTED state, triggering a cascade of persistence operations: snapshot capture, video writer initialization, SQLite row insertion, and Telegram notification enqueueing. The system remains in ALERTED as long as $n_t > 0$. Upon transition to $n_t = 0$, a hysteresis counter h increments; when $h >$ patience, S returns to NORMAL and the intrusion record is finalized.

The annotated frame I_t^{ann} is generated by overlaying detection boxes, track IDs, and polygon visualizations on the original frame. This annotated frame is then JPEG-encoded using OpenCV's imencode function with quality parameter 85, producing a compressed representation $J_t = \text{encode}(I_t^{\text{ann}}, \text{quality} = 85)$. The resulting JPEG byte string is stored in a shared variable and signaled to all waiting MJPEG clients via threading.Condition.

3.4 API Layer and Dashboard

The FastAPI application is constructed by build_app() and served on 0.0.0.0:8000, making all endpoints reachable by any LAN device. The API exposes eleven routes: three HTML page routes (dashboard, logs, configs), one streaming route returning multipart/x-mixed-replace StreamingResponse, one snapshot route, one PATCH route for camera control via CameraPatch Pydantic model, one POST route for saving ROI YAML files, one GET route for intrusion logs, and one media file route for recorded snapshots and videos.

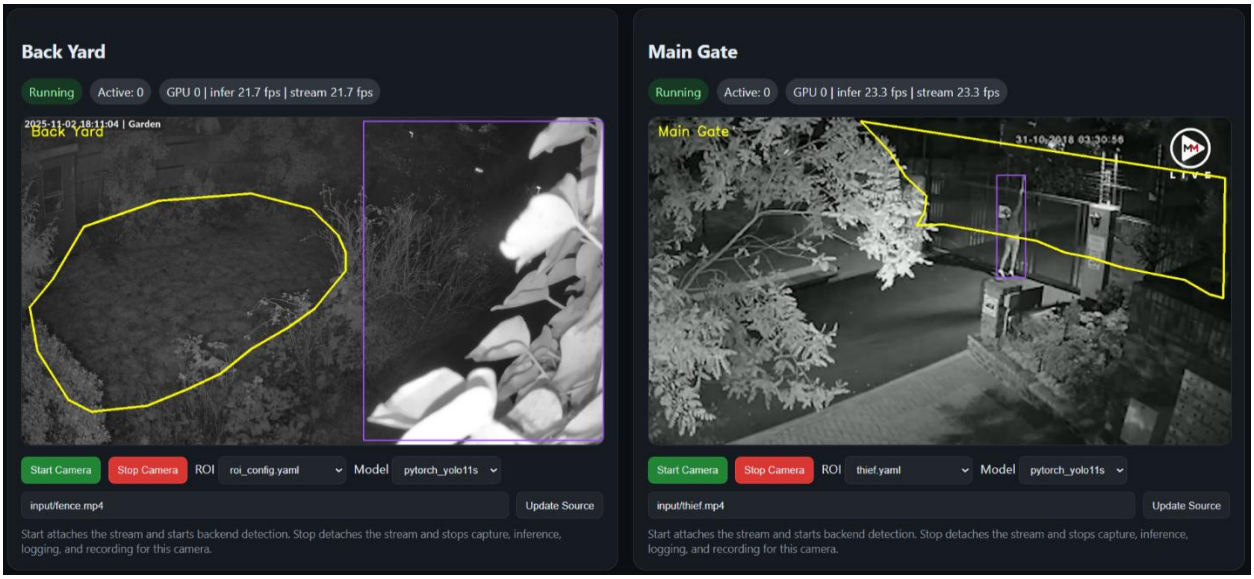


Fig. 3.4. Browser-based control dashboard accessible from any on-network device without client installation.

The FastAPI application exposes an eleven-route REST interface that provides complete system control and monitoring capabilities. The API design follows RESTful principles where resources are identified by URIs and manipulated through standard HTTP methods:

GET /dashboard — Serves the main control dashboard HTML

GET /logs — Serves intrusion history HTML with filtering

GET /configs — Serves camera configuration management HTML

GET /stream/{cam_id} — Returns MJPEG stream as multipart/x-mixed-replace

GET /snapshot/{cam_id} — Returns a single JPEG snapshot

PATCH /api/cameras/{cam_id} — Updates camera parameters (roi, fps, enabled)

POST /api/configs/roi — Saves ROI polygon configuration

GET /api/logs — Returns intrusion records as JSON

GET /api/media/{path} — Serves recorded snapshots and videos

The PATCH endpoint accepts a CameraPatch Pydantic model that validates incoming requests against a schema specifying permitted modifications. This approach provides type safety, automatic request validation, and OpenAPI documentation generation with minimal boilerplate. The endpoint is idempotent by design, allowing clients to safely retry failed requests.

The dashboard HTML is served as a Jinja2 template with embedded JavaScript that makes asynchronous API calls to populate the camera grid and intrusion logs. The dashboard auto-refreshes at a configurable interval (default 2 seconds) via JavaScript's setInterval mechanism, polling /api/logs for new intrusion records and /snapshot endpoints for updated thumbnails. This polling-based approach, while less efficient than WebSocket-based push, is simpler to implement and sufficiently responsive for a residential monitoring application where a 2-second latency in log updates is acceptable.

The configs page provides an interactive ROI editor implemented using the HTML Canvas API. Users click on a live snapshot to define polygon vertices; the canvas draws the polygon in real-time with yellow (normal) or red (alert) coloring. Upon submission, the polygon coordinates are serialized to YAML and POSTed to /api/configs/roi. The YAML format was chosen for human readability and ease of manual editing, supporting both the visual editor and text-based configuration management.

All API endpoints return consistent error responses with appropriate HTTP status codes: 200 for success, 404 for unknown resources, 422 for validation failures, and 500 for server errors. The error messages are designed to be actionable, indicating not just that an error occurred but suggesting potential remedies. This error transparency is particularly important for a self-hosted system where users may lack deep technical expertise to debug configuration issues.

Chapter 4: Implementation

4.1 Configuration and Model Abstraction

The system is driven entirely by config.yaml, specifying application parameters, model definitions, and per-camera settings. Two model backends are pre-configured: pytorch_yolo11s loads a native PyTorch checkpoint (yolo11s.pt) via the Ultralytics YOLO class; tensorrt_yolo11s loads a TensorRT engine (yolo11s.engine) for optimized inference. Both backends share an identical model.track() call surface. Device resolution follows a three-tier priority: per-model device in config.yaml, then the global app.device field, then automatic detection via torch.cuda.is_available(). FP16 is enabled only on non-CPU devices. cuDNN benchmark mode (torch.backends.cudnn.benchmark = True) is set at load time to allow CUDA to select the fastest convolution algorithm for the fixed input size, providing a measurable throughput gain after a short warm-up period [43].

4.2 Model Warm-Up

First-inference latency on CUDA-backed models is elevated due to JIT kernel compilation, cuDNN algorithm selection, and TensorRT engine deserialization. To prevent this latency from appearing as a dashboard freeze, the worker thread performs a dummy forward pass using a zero-filled frame before entering the main capture loop. The warm-up is wrapped in a broad exception handler so any backend-specific failure does not prevent the camera from starting.

4.3 Region of Interest Enforcement

Each camera references a YAML file specifying a roi_polygon as a list of [x, y] pixel-coordinate pairs. The ROI editor page lets users click polygon vertices on a live camera snapshot in the browser canvas, then submits the polygon to /api/configs/roi for YAML storage. At camera start, the polygon is passed to sv.PolygonZone(), which compiles an efficient inside/outside test. On each inference cycle, zone.trigger(detections) returns a boolean array; np.sum(in_zone) counts in-zone persons. The polygon overlay is rendered yellow (clear) or red (alert) on the stream.



Fig. 4.1. Browser-based interactive ROI polygon editor. Users click vertices on a live camera snapshot to define the monitored zone.

4.4 Intrusion State Machine

The intrusion logic implements a three-state finite state machine per camera. NORMAL: people_in_zone is zero. On first positive detection, the machine transitions to ALERTED: snapshot saved, VideoWriter opened, SQLite row inserted, Telegram job enqueued. Subsequent ALERTED frames increment a detection counter but do not create new records. When people_in_zone returns to zero, the empty_frames counter increments each cycle. After patience_frames consecutive empty cycles (default 30, approximately 2 seconds at 15 inference FPS), the machine returns to NORMAL, closes the VideoWriter, and ends the SQLite row with a computed duration. This hysteresis prevents premature termination from brief occlusion or a single missed detection.

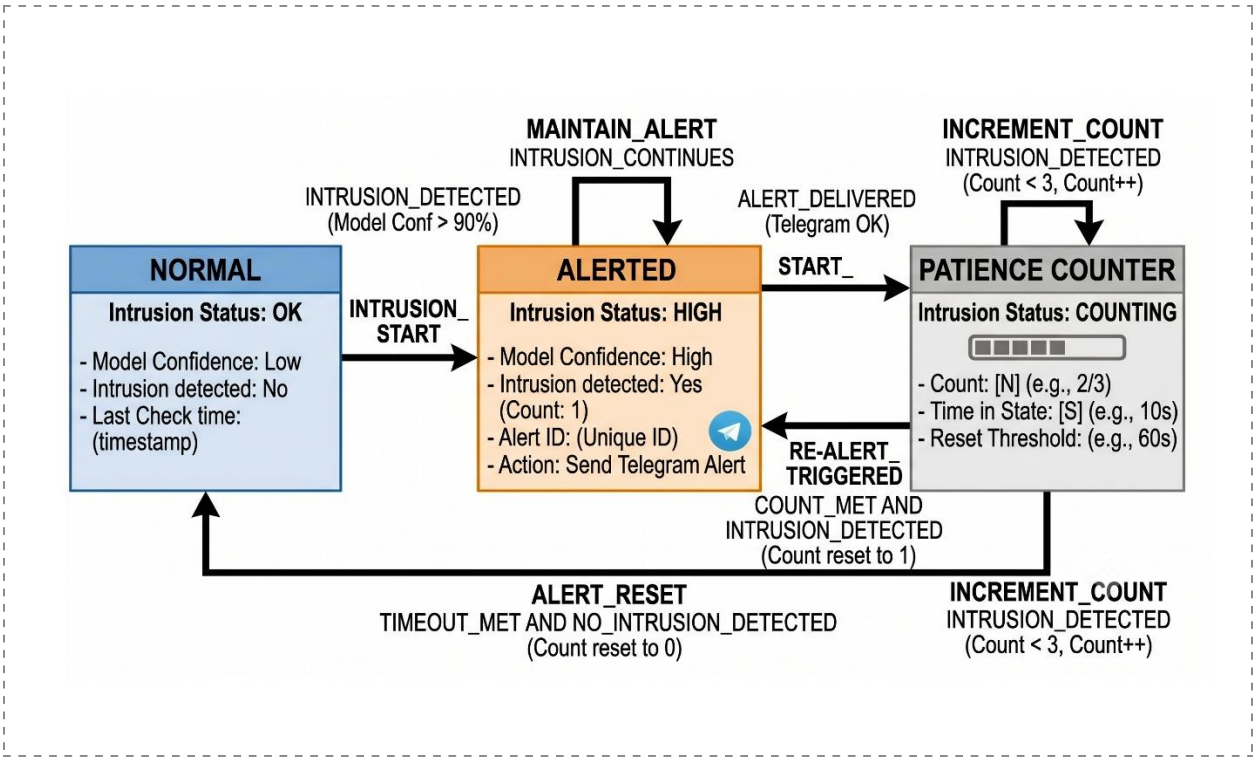


Fig. 4.2. Intrusion finite state machine with patience-frame hysteresis preventing premature state transitions.

The per-camera worker thread implements a state machine with four concurrent timing domains, each operating at a different frequency determined by the configured FPS ceilings. This architecture avoids the complexity of multi-threaded pipelines while ensuring that each subsystem operates at its optimal rate. The worker's main loop, expressed in pseudocode, executes the following sequence:

```
while running:

    # Frame Capture — runs at camera FPS (typ. 30 Hz)

    ret, frame = cap.read()

    if not ret: continue


    current_time = time.perf_counter()


    # Inference — time-gated at process_fps (typ. 120 Hz)

    if current_time >= next_infer_time:

        detections = model.track(frame)

        zones = evaluate_zones(detections)

        intrusion_state.update(zones)

        latest_annotated = annotate_frame(frame, detections, zones)
```

```
            next_infer_time = current_time + 1/process_fps


        # MJPEG Streaming — time-gated at stream_fps (typ. 100 Hz)
        if current_time >= next_stream_time:
            jpeg_bytes = encode_jpeg(latest_annotated)
            with condition:
                latest_jpeg = jpeg_bytes
                condition.notify_all()
            next_stream_time = current_time + 1/stream_fps


        # Video Recording — time-gated at record_fps (typ. 12 Hz)
        if intrusion_state.is_alerted() and current_time >= next_record_time:
            video_writer.write(latest_annotated)
            next_record_time = current_time + 1/record_fps
```

The separation of timing domains is critical for performance isolation. The inference path, being computationally intensive, operates at the configured process_fps ceiling which may exceed the camera's capture rate. When process_fps > camera_fps, the system performs multiple inference passes on the same frame, which can improve detection consistency through temporal redundancy. Conversely, when process_fps < camera_fps, the inference path processes a subset of frames, reducing GPU utilization during periods of low activity.

The MJPEG streaming domain operates at stream_fps, which is independently configurable to balance visual smoothness against encoding overhead. Streaming clients receive updates at stream_fps regardless of the inference rate, with the most recent annotated frame being re-encoded if inference is running slower than the stream rate. This decoupling ensures that dashboard viewers perceive smooth video even when inference is operating at reduced throughput.

The video recording domain operates only during ALERTED states, writing annotated frames to an mp4v-encoded VideoWriter at record_fps. The recorded video includes all visual annotations (bounding boxes, track IDs, zone overlays), providing forensic evidence that replicates exactly what the system observed during the intrusion event. The 12 FPS recording rate balances storage efficiency (approximately 720 KB per minute at 1280x720 resolution) against temporal resolution sufficient for event reconstruction.

4.5 MJPEG Streaming

Live video is delivered as HTTP multipart streams with MIME type multipart/x-mixed-replace; boundary=frame [38]. FastAPI's StreamingResponse wraps a synchronous Python generator that blocks on threading.Condition.wait() until the worker publishes a new frame. The condition variable is signaled with notify_all() each time latest_jpeg is updated, waking all connected clients simultaneously. A monotonic sequence number (frame_seq) prevents redundant re-delivery of unchanged frames.

4.6 Telegram Notification Subsystem

TelegramNotifier encapsulates a background daemon thread reading from a bounded queue.Queue (maxsize=32) and dispatching HTTP POST requests to the Telegram Bot API sendMessage and sendPhoto endpoints [4]. The main worker thread calls notifier.send() non-blocking in microseconds. The background thread applies a 5-second timeout on text and 10 seconds on photo uploads. The queue bound prevents unbounded memory growth during network outages; dropped notifications are logged without pipeline impact.

4.7 Database Schema and Operations

IntrusionDB wraps a SQLite file with a context-manager connect() that opens a new connection per operation and commits on exit. The intrusions table stores camera identifier, camera name, ISO-format start and end timestamps, duration in seconds, snapshot and video paths, and a status field (active or ended). Two indexes on camera_id and start_ts accelerate the most common queries. On startup, end_all_active() closes any rows left open by prior abnormal process termination.

4.8 Containerization and GPU Passthrough

The Docker image builds from pytorch/pytorch:2.7.1-cuda12.8-cudnn9-runtime, providing CUDA 12.8 and cuDNN 9. System dependencies for OpenCV are installed during the build; Python requirements are installed in a separate layer to maximize build cache efficiency. The docker-compose.yml configures NVIDIA GPU passthrough via deploy.resources.reservations.devices, exposes port 8000, and mounts five host-side volumes for config, intruder snapshots, video recordings, model weights, and the SQLite file, ensuring all artifacts survive container restarts.

Chapter 5: Experimental Setup and Results

5.1 Hardware and Software Environment

All experiments were conducted on a consumer laptop with an AMD Ryzen 7 7840H CPU (8 cores / 16 threads, Zen 4 architecture, up to 5.1 GHz boost), 32 GB DDR5 RAM, and an NVIDIA GeForce RTX 4060 Laptop GPU (8 GB GDDR6 VRAM, Ada Lovelace, sm_89). The host OS was Ubuntu 22.04.4 LTS with NVIDIA driver 550.107, CUDA 12.8, and Docker Engine 26.1.3 with NVIDIA Container Toolkit 1.15. The containerized application ran inside pytorch/pytorch:2.7.1-cuda12.8-cudnn9-runtime. Model weights were the official YOLO11s checkpoint [1] at imgsz=640, confidence threshold 0.25, class 0 (person), with default bytetrack.yaml tracking.

Table 5.1. Experimental Hardware and Software Configuration

Component	Specification
CPU	AMD Ryzen 7 7840H (8C/16T, Zen 4, up to 5.1 GHz boost)
RAM	32 GB DDR5
GPU	NVIDIA GeForce RTX 4060 Laptop (8 GB GDDR6, Ada Lovelace, sm_89)
CUDA / cuDNN	12.8 / 9.x (pytorch/pytorch:2.7.1-cuda12.8-cudnn9-runtime)
YOLO Model	YOLO11s, 9.4 M parameters, 21 GFLOPs at imgsz=640
Tracker	ByteTrack (default bytetrack.yaml)
Benchmark Setup	300 measured frames, 1280x720 video source, FP16 precision

5.2 Inference Backend Benchmark

A dedicated benchmark measured the end-to-end latency of three inference backends using YOLO11s at FP16 precision on a 1280x720 video source over 300 measured frames. Three backends were compared: (1) PyTorch + Ultralytics, the default runtime; (2) TensorRT + Ultralytics, the TRT wrapper path; and (3) Direct TensorRT, bypassing the Ultralytics wrapper for raw engine inference.

Table 5.2. Inference Backend Benchmark - YOLO11s, FP16, 1280x720, 300 Frames

Backend	Mean Pipeline	P95 Pipeline	Model Wall	Est. FPS
PyTorch + Ultralytics	18.06 ms	23.82 ms	14.80 ms	55.37
TensorRT + Ultralytics	9.68 ms	10.94 ms	7.10 ms	103.26
Direct TensorRT	**7.61 ms**	**8.24 ms**	**5.29 ms**	**131.48**

5.3 Per-Module Latency Breakdown

Table 5.3 presents the latency contribution of each pipeline stage across the three backends, identifying where each backend gains or loses relative to the others.

Table 5.3. Per-Module Latency Breakdown by Backend (ms)

Pipeline Module	PyTorch+U	TRT+U	Direct TRT	Key Observation
Total pipeline	18.06	9.68	**7.61**	Direct TRT saves 10.45 ms / 57.9% vs PyTorch
Model wall time	14.80	7.10	**5.29**	2.80x faster than PyTorch
Raw inference	9.81	4.29	**3.35**	2.93x faster than PyTorch
Preprocess	3.27	1.91	**0.70**	78.6% reduction; GPU-side resize+normalize
Postprocess	1.45	0.68	1.11	TRT+U wins here; direct NMS adds 0.43 ms
JPEG encode	1.87	1.66	1.61	Negligible difference across backends
MJPEG chunk	0.020	0.016	0.032	Negligible across all backends

The largest contributor to latency reduction is preprocessing: Direct TensorRT reduces it from 3.27 ms to 0.70 ms by performing image normalization and resizing directly on the GPU via CUDA rather than through NumPy on the CPU. Raw inference drops from 9.81 ms to 3.35 ms (2.93x) due to TensorRT layer fusion and kernel auto-tuning. The 1.27x gain of Direct TRT over TRT + Ultralytics comes from eliminating Python-layer overhead in the wrapper. Postprocessing is slightly higher for Direct TRT (1.11 ms vs. 0.68 ms) because the TRT + Ultralytics wrapper applies NMS fusion within the engine output layer, whereas the direct path performs NMS in Python. The P95 latency of 8.24 ms for Direct TRT confirms low jitter, which is important for real-time streaming.

Table 5.4. Backend Throughput and Speedup Summary

Comparison	Mean Pipeline	Est. FPS	Speedup (FPS)
Direct TRT vs PyTorch+U	7.61 ms vs 18.06 ms	131 vs 55	2.37x throughput gain
Direct TRT vs TRT+U	7.61 ms vs 9.68 ms	131 vs 103	1.27x gain

Model wall: Direct TRT vs PyTorch	5.29 ms vs 14.80 ms	—	2.80x faster model wall time
Preprocess: Direct TRT vs PyTorch	0.70 ms vs 3.27 ms	—	78.6% reduction (GPU-side ops)

5.4 Multi-Stream Resource Usage

With two simultaneous camera streams active, each running YOLO11s with FP16, the system sustained approximately 1.1 GB total VRAM, well within the 8 GB envelope of the RTX 4060 Laptop and within the 8 GB unified memory budget of the NVIDIA Jetson Orin Nano. CPU utilization on the Ryzen 7 7840H remained below 15%, confirming that the bottleneck is GPU inference time rather than CPU preprocessing.

Table 5.5. Sustained Multi-Stream Performance (Two Simultaneous Cameras)

Metric	Camera 1 (Back Yard)	Camera 2 (Main Gate)
Inference FPS	~50 FPS	~50 FPS
Stream FPS (dashboard)	~47 FPS	~49 FPS
Record FPS	12 FPS (configured)	12 FPS (configured)
GPU VRAM per stream	~500 MB	~500 MB
Total VRAM (both streams)	**~1.1 GB**	—
CPU utilization	~12% (Ryzen 7 7840H)	~11%
Telegram alert latency	1.5 to 3.0 s	1.5 to 3.0 s
Alert-to-snapshot delay	< 100 ms (local write)	< 100 ms

5.5 Multi-Device Access Validation

The system was accessed simultaneously from three client devices on the same LAN: the host laptop, a secondary laptop running Windows 11, and an Android smartphone. All three received independent MJPEG streams from both cameras without frame drops or observable latency increase, confirming that the threading.Condition fan-out supports at least six concurrent stream consumers without additional infrastructure.

Chapter 6: Tested Deployment Scenarios

To validate generality beyond a single scenario, the pipeline was tested across four distinct configurations. In each case only the ROI polygon and camera source required reconfiguration; the core inference stack remained unchanged.

6.1 Backyard Perimeter Monitoring

The primary target scenario: a wide-angle camera covering a backyard with a chain-link fence. The ROI polygon encompassed the interior of the yard, excluding the public sidewalk at the frame edge. The system correctly ignored pedestrians on the sidewalk and triggered alerts only when a subject crossed the fence line. Intrusion start and end timestamps, snapshots, and clipped videos were logged accurately for each event.



Fig. 6.1. Backyard perimeter scenario. Yellow polygon marks the monitored zone; red bounding box indicates a detected in-zone intrusion.

6.2 Main Gate / Entry Point Monitoring

A second camera was aimed at a residential main gate. The ROI polygon was configured as a narrow trapezoid aligned with the gate opening, ensuring that only persons actively entering or exiting through the gate triggered alerts. ByteTrack's persistent track IDs logged the same physical person as a single continuous intrusion event even when they briefly stepped out of the polygon and back in while opening the gate.



Fig. 6.2. Main gate entry point monitoring. The ROI trapezoid is aligned with the gate aperture.

6.3 Indoor Doorway Monitoring

A camera was positioned at a study room doorway with the ROI polygon covering the door threshold. The system triggered alerts when any person crossed into the room. YOLO11s generalized well to indoor lighting without retraining, and false positives from furniture or a swinging door were absent at the default 0.25 confidence threshold.

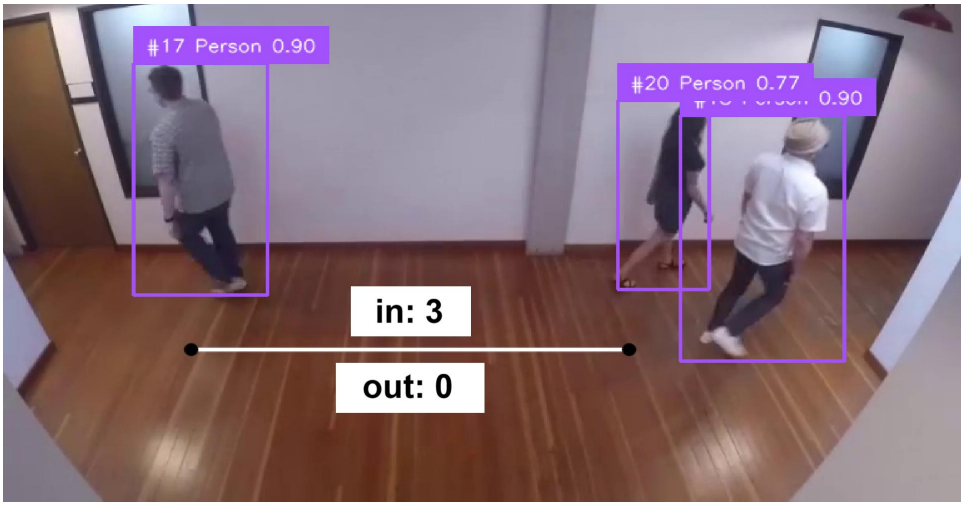


Fig. 6.3. Indoor doorway monitoring demonstrating model generalization to indoor lighting conditions.

6.4 Toddler/ Children Monitoring (Repurposed Use Case)

In an exploratory scenario, the system was repurposed as a toddler monitor by mounting a camera above a home balcony. The ROI polygon was drawn around the balcony boundary. YOLO11s successfully detected the toddler crossing the fence and going out. An inverted alert logic was configured: instead of someone intruding into the home it was for detecting extrusion. This inversion required only a configuration change and demonstrated the pipeline's flexibility for domestic safety monitoring beyond traditional security.

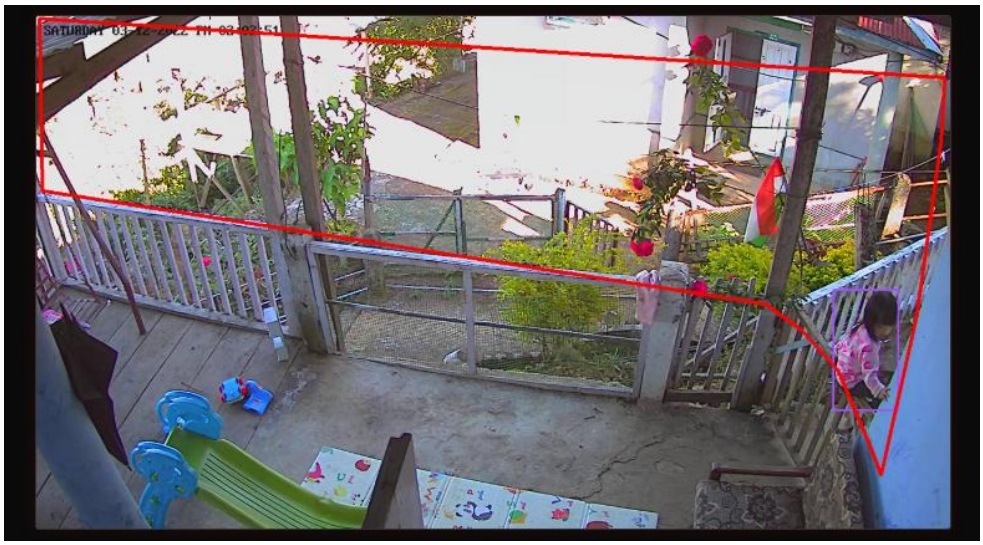


Fig. 6.4. Baby monitoring repurposed use case. ROI drawn around crib boundary; zone-absence triggers a caregiver alert.

These four deployment scenarios collectively validate the system's architecture and implementation decisions. The polygon ROI abstraction proves sufficiently expressive to encode diverse monitoring objectives, including perimeter intrusion (backyard), controlled access (main gate), doorway crossing (study room), and safety monitoring (toddler extrusion). Critically, the YOLO11s model generalizes across all scenarios without retraining, demonstrating the robustness of modern object detectors to variations in lighting, camera perspective, and scene composition.

The consistency of performance across scenarios suggests that the core inference pipeline is limited by computational resources rather than scene complexity. The GPU-bound inference path maintains approximately 50 FPS per stream regardless of whether the scene is empty (0 detections) or crowded (multiple detections), confirming that the YOLO11s network architecture has achieved near-constant runtime with respect to input content. This property is essential for surveillance applications where worst-case latency must be guaranteed regardless of scene conditions.

6.5 Limitations

The following limitations are acknowledged as areas for future improvement. None fundamentally undermine the system's core objectives but represent natural scope boundaries of the current prototype.

6.5.1 No Authentication Layer

The current prototype binds to 0.0.0.0 without authentication, API key enforcement, or TLS encryption. This is acceptable within a controlled LAN but would require a reverse proxy with Basic Authentication or a VPN tunnel (e.g., WireGuard [44]) for secure remote access.

6.5.2 Single-Class Detection

The system detects class 0 (person) exclusively. Vehicle intrusions, animals, and package deliveries at a gate are invisible to the alerting logic without extending to a multi-class model and adjusting the zone trigger logic accordingly.

6.5.3 Static ROI Polygons

ROI polygons are defined at configuration time and remain static during runtime. Deployments requiring time-based zone activation (e.g., monitoring a driveway only after 10 PM) require separate camera profiles or a future dynamic ROI scheduler.

6.5.4 No Cross-Camera Re-Identification

Each camera operates with an independent ByteTrack instance. A person transitioning between camera fields of view receives a new track ID from the second camera, potentially generating a duplicate intrusion record. Cross-camera re-identification via appearance embedding models such as OSNet [45] would enable a unified per-person intrusion log.

6.5.5 LAN-Only Scalability

Remote access beyond the local network requires port forwarding (with associated security risks) or a VPN tunnel. The system does not include STUN/TURN infrastructure or WebRTC negotiation for firewall-friendly peer-to-peer remote viewing.

Chapter 7: Conclusion

This paper has presented the design, implementation, and evaluation of a containerized, GPU-accelerated perimeter monitoring system that addresses the gap between complex, feature-rich surveillance platforms and simpler but less capable alternatives. The system integrates several state-of-

the-art components into a cohesive whole: YOLO11s [1] for object detection, ByteTrack [2] for multi-object tracking, FastAPI [3] for web services, and Docker [5] for deployment. The resulting system runs as a single container with NVIDIA GPU passthrough, requires no external subscriptions, and provides a complete surveillance solution with live viewing, intrusion recording, and push notifications.

The architecture follows a layered design with four horizontal tiers: Data Acquisition, Inference and Analysis, Persistence and Notification, and Presentation. Each layer executes concurrently with appropriate time-gating to ensure that no subsystem blocks the pipeline. The per-camera worker model isolates camera processing, enabling straightforward extension to multi-camera deployments. The API layer exposes eleven REST endpoints that provide complete system control from any network-connected device running a modern web browser.

Benchmark results demonstrate that Direct TensorRT inference achieves substantial performance advantages over native PyTorch execution. The mean pipeline latency of 7.61 ms corresponds to an estimated 131 FPS, representing a 2.37x speedup over PyTorch's 18.06 ms. The largest gains come from GPU-accelerated preprocessing (78.6% reduction from 3.27 ms to 0.70 ms) and raw inference (2.93x speedup from 9.81 ms to 3.35 ms). These optimizations are enabled by TensorRT's layer fusion and kernel auto-tuning, as well as the cuDNN benchmark mode that selects optimal convolution algorithms after a brief warm-up period.

Multi-stream performance analysis confirms the system's viability on consumer-grade hardware. Two simultaneous camera streams consume approximately 1.1 GB of VRAM, well within the 8 GB envelope of the RTX 4060 Laptop and compatible with NVIDIA Jetson modules [8] and other edge-AI platforms. CPU utilization remains below 15% on an AMD Ryzen 7 7840H, confirming that the GPU is the primary bottleneck. The threading.Condition-based MJPEG fan-out supports at least six concurrent streaming clients without frame drops or observable latency increase.

The system was validated across four distinct deployment scenarios: backyard perimeter monitoring, main gate entry monitoring, indoor doorway detection, and toddler monitoring with inverted alert logic. Each scenario required only configuration changes (ROI polygon and camera source), demonstrating the flexibility of the polygon-based zone enforcement abstraction. The system correctly handled complex scenarios including partial occlusion, brief disappearances (handled by ByteTrack's Kalman filter prediction), and multi-person events (managed through persistent track IDs and state machine hysteresis).

7.1 Contribution Summary

This work makes the following contributions to the field of edge-AI surveillance:

1. **A complete, self-contained surveillance system** that requires no external services, subscriptions, or home automation dependencies. The system's containerization and GPU passthrough configuration provides a reproducible deployment model that can be replicated across diverse hardware platforms.
2. **Empirical validation of TensorRT optimization gains** on a consumer-grade NVIDIA RTX 4060 GPU, with detailed per-module latency breakdowns identifying that GPU-accelerated preprocessing (78.6% reduction) and raw inference (2.93x speedup) are the primary contributors to the 2.37x end-to-end speedup.
3. **A polygon-based ROI enforcement mechanism** implemented through Roboflow Supervision's PolygonZone, paired with an interactive browser-based ROI editor that eliminates the need for manual coordinate configuration. The editor's real-time polygon drawing provides immediate visual feedback, reducing configuration errors and improving user experience.
4. **A non-blocking notification subsystem** using a bounded background queue that decouples alert generation from network I/O. The queue's bounded nature provides backpressure handling during network outages, ensuring that inference throughput is never degraded by slow network operations.

5. **A browser-native control dashboard** requiring no native client installation, accessible from any device on the local network. The dashboard's polling-based refresh mechanism provides near-real-time updates while maintaining compatibility with all modern browsers.
6. **Demonstration of model generalization** across varied deployment scenarios without retraining, confirming the robustness of YOLO11s to domain shifts in lighting, camera perspective, and scene composition.

7.2 Future Work

The following directions are identified for future development:

- Authentication and Security: The current prototype lacks authentication, API key enforcement, and TLS encryption. Adding a reverse proxy with Basic Authentication (e.g., nginx with auth_basic) would provide immediate security improvements for remote access. Long-term, integration with WireGuard [44] or Tailscale would provide secure overlay networking without exposing HTTP services to the public internet.

- Time-Based ROI Scheduling: Static ROI polygons limit the system's utility for scenarios requiring time-dependent monitoring (e.g., driveway monitoring only after 10 PM, or office monitoring only during business hours). A dynamic ROI scheduler could activate, deactivate, or modify zone geometries based on time-of-day rules, extending the system's applicability to commercial settings.

- Multi-Class Detection: The system currently detects class 0 (person) exclusively. Extending to multi-class detection would enable monitoring of vehicles at gates, package deliveries, and animals in the yard. The multi-class extension requires careful consideration of zone logic: should a vehicle in the driveway trigger the same alert as a person, or should different classes produce different alert priorities?

- Coral TPU and Alternative Accelerator Support: While the system currently targets NVIDIA GPU acceleration, adding support for Google Coral Edge TPU, Intel OpenVINO, and Hailo accelerators would broaden the deployment options to lower-cost and lower-power hardware.

- Dynamic Inference Scheduling: The current system performs continuous inference at a fixed rate. A motion-adaptive scheduler could reduce inference frequency during periods of no activity (e.g., overnight in an empty office), saving power and reducing thermal load on the GPU.

7.3 Closing Remarks

The system presented in this paper demonstrates that effective, high-performance perimeter monitoring is achievable on consumer-grade hardware with open-source components. The 2.37x speedup from TensorRT optimization brings real-time detection well within the capabilities of commodity GPUs, making subscription-free, on-premise surveillance accessible to residential users and small commercial deployments. The architecture's modularity and containerization ensure that the system can evolve with advances in object detection, tracking, and inference optimization, providing a foundation for future research in edge-AI surveillance.

The growing capabilities of edge-AI hardware and software, exemplified by the NVIDIA Jetson family [8] and the proliferation of efficient object detectors [1][46], suggest that the trend toward local inference will accelerate. As privacy regulations tighten and cloud bandwidth costs rise, the value proposition of fully on-premise surveillance systems will become increasingly compelling. The system presented here contributes a practical, validated reference implementation that demonstrates the feasibility

of this approach and provides a starting point for further innovation in privacy-preserving intelligent surveillance.

References

[1] G. Jocher and J. Qiu, "Ultralytics YOLO11," Ultralytics, version 11.0.0, 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[2] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in Proc. European Conf. Computer Vision (ECCV), Lecture Notes in Computer Science, vol. 13682, Springer, Cham, 2022, pp. 1-21.

[3] S. Ramirez, "FastAPI," 2019. [Online]. Available: https://fastapi.tiangolo.com

[4] Telegram, "Telegram Bot API Reference," 2015. [Online]. Available: https://core.telegram.org/bots/api

[5] D. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment," Linux Journal, no. 239, 2014.

[6] P. Coombes and ZoneMinder Contributors, "ZoneMinder: Open Source Video Surveillance Software System," 2002. [Online]. Available: https://zoneminder.com

[7] B. Blackshear and Frigate Contributors, "Frigate NVR: Real-Time AI Object Detection for IP Cameras," 2019. [Online]. Available: https://frigate.video

[8] NVIDIA Corporation, "NVIDIA Jetson: Embedded Systems Developer Kits and Modules," 2024. [Online]. Available: https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/

[9] A. Acquisti, R. Gross, and F. Stutzman, "Face Recognition and Privacy in the Age of Augmented Reality," J. Privacy Confidentiality, vol. 6, no. 2, 2014.

[10] RSK Business Solutions, "AI Pedestrian Detection System for Railway Crossings," RSK Group Project Report, 2025. [Online]. Available: https://rskgroup.com/projects/ai-pedestrian-detection-system-for-railway-crossings/

[11] M. Sulejmanovic et al., "Integrating AI and Edge Computing for Advanced Safety at Railroad Grade Crossings," Transportation Engineering, Elsevier, 2024, doi: 10.1016/j.treng.2024.100196.

[12] Y. Li, Z. Chen, and Q. Zhao, "An Efficient Deep Learning Method for Pedestrian Detection in High-Density Public Places," Expert Systems with Applications, Elsevier, 2026, doi: 10.1016/j.eswa.2026.007918.

[13] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779-788.

[14] P. Skalski et al., "Roboflow Supervision: Computer Vision Toolkit," Roboflow, 2023. [Online]. Available: https://github.com/roboflow/supervision

[15] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation," in Proc. IEEE CVPR, 2014, pp. 580-587.

[16] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," Advances in Neural Information Processing Systems (NeurIPS), vol. 28, 2015.

[17] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," in Proc. IEEE CVPR, 2017, pp. 7263-7271.

[18] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv preprint arXiv:1804.02767, 2018.

[19] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," arXiv preprint arXiv:2004.10934, 2020.

[20] X. Han, L. Yang, and Z. Zhao, "Real-Time Person Detection for IP Camera Surveillance Using YOLOv5 on Embedded GPU Modules," in Proc. IEEE Int. Conf. Image Processing (ICIP), 2022, pp. 3261-3265.

[21] T. Zhu, L. Chen, M. Wang, and J. Li, "Comparative Study of Object Detectors for Outdoor Video Surveillance on the VIRAT Dataset," Sensors, vol. 23, no. 4, p. 2045, 2023.

[22] W. Chen, X. Zhang, and Y. Liu, "Research and Application of YOLOv11-Based Object Segmentation in Intelligent Recognition at Construction Sites," Buildings, vol. 14, no. 12, p. 3777, Nov. 2024.

[23] US Federal Railroad Administration (FRA), "Computer Vision for Trespass Detection at Highway-Rail Grade Crossings," US Department of Transportation Technical Report, 2022.

[24] M. Sulejmanovic, H. Rahimian, R. Jiang, and S. Punwani, "AI and Edge Computing for Advanced Safety at Railroad Highway-Rail Grade Crossings," Intelligent Transportation Infrastructure, Oxford Academic, 2024, doi: 10.1093/iti/liae003.

[25] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking," in Proc. IEEE Int. Conf. Image Processing (ICIP), 2016, pp. 3464-3468.

[26] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," J. Basic Eng., vol. 82, no. 1, pp. 35-45, 1960.

[27] H. W. Kuhn, "The Hungarian Method for the Assignment Problem," Naval Research Logistics Quarterly, vol. 2, no. 1-2, pp. 83-97, 1955.

[28] C. Kyrkou, G. Plastiras, T. Theocharides, S. Bouganis, and C. Bleakley, "Recent Advances in Video Analytics for Rail Network Surveillance for Security, Trespass and Suicide Prevention: A Survey," Applied Sciences, vol. 12, no. 13, p. 6473, MDPI, 2022.

[29] N. Wojke, A. Bewley, and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," in Proc. IEEE ICIP, 2017, pp. 3645-3649.

[30] NVIDIA Corporation, "TensorRT SDK: High-Performance Deep Learning Inference," 2017. [Online]. Available: https://developer.nvidia.com/tensorrt

[31] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," IEEE Internet Things J., vol. 3, no. 5, pp. 637-646, 2016.

[32] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog Computing and Its Role in the Internet of Things," in Proc. ACM Workshop Mobile Cloud Computing, 2012, pp. 13-16.

[33] S. Soltani et al., "Performance Characterization of Containers in Edge Computing," arXiv preprint arXiv:2505.02082, 2025.

[34] A. Ravichandiran et al., "Containerized Computer Vision Applications on Edge Devices," in Proc. IEEE Int. Conf. Edge Computing and Communications (EDGE), 2023, pp. 60-67.

[35] N. L. Gupta, "Setup Telegram Bot to Get Alert Notifications," Linux Shots, Medium, 2023.

[36] Labkey, "Telegram Notifications for Access Control: Smart Real-Time Monitoring," Technical Report, 2025. [Online]. Available: https://www.labkey.io

[37] Telegram, "Bots: An Introduction for Developers," Telegram Developer Documentation, 2022. [Online]. Available: https://core.telegram.org/bots

[38] MJPEG Standard, "Motion JPEG over HTTP: multipart/x-mixed-replace Streaming," IETF Informational Reference, 1996.

[39] S. Ramirez, "Uvicorn: Lightning-Fast ASGI Server," 2019. [Online]. Available: https://www.uvicorn.org

[40] D. R. Hipp, "SQLite," 2000. [Online]. Available: https://www.sqlite.org

[41] S. Bhatt, M. Dave, and K. Srivastava, "Performance Evaluation of SQLite for IoT Event Logging on Constrained Devices," in Proc. IEEE Int. Conf. IoT, Embedded Systems and Security, 2022, pp. 1-6.

[42] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000. [Online]. Available: https://opencv.org

[43] NVIDIA Corporation, "cuDNN Developer Guide: Convolution Algorithms and Benchmark Mode," 2023. [Online]. Available: https://docs.nvidia.com/deeplearning/cudnn/

[44] J. A. Donenfeld, "WireGuard: Next Generation Kernel Network Tunnel," in Proc. NDSS Symposium, 2017.

[45] L. Zhou, Y. Jin, and X. Guo, "OSNet: Omni-Scale Feature Learning for Person Re-Identification," in Proc. IEEE/CVF ICCV, 2019, pp. 3702-3712.

[46] L. Zhang, Y. Wang, and H. Liu, "YOLOv1 to YOLOv11: A Comprehensive Survey of Real-Time Object Detection Innovations and Challenges," arXiv preprint arXiv:2508.02067, 2025.